



## Today's agenda

- ↳ Count sort  $\rightarrow O(N)$
- ↳ wave form of array
- ↳ Sorting an array of objects
- ↳ Subsequence



# AlgoPrep



## Count sort

↳ Sort the array using Count sort.

Ex: arr[]: { 2 5 3 0 2 3 0 3 }

Step 1: Figure out max of the array.

↳ max = 5

Step 2: Count occurrence of each number.

| Count | 0 | 1 | 2 | 3 | 4 | 5 | ← No.  |
|-------|---|---|---|---|---|---|--------|
|       | 2 | 0 | 2 | 3 | 0 | 1 | ← freq |

Step 3: do the sorting step.

arr[]: { ~~2~~ 0 ~~5~~ 0 ~~2~~ 2 ~~3~~ 3 ~~3~~ 3 ~~5~~ } ✓



// Pseudo code

```
void CountSort (int arr[N]) {  
    int max = -∞;  
    for (int i = 0; i < N; i++) {  
        if (arr[i] > max) { max = arr[i]; }  
    }
```

T.C:  $O(N + N + \text{max})$

$= O(N + \text{max})$

↑  
arr.length    ↑  
                  max value  
                  of array

S.C:  $O(\text{max})$

```
int[] count = new int[max + 1];
```

```
for (int i = 0; i < N; i++) {  
    int val = arr[i];  
    count[val]++;  
}
```

```
int k = 0;
```

```
for (int i = 0; i <= max; i++) {  
    int c = count[i];  
    for (int j = 0; j < c; j++) {  
        arr[k] = i; k++;  
    }  
}
```

```
}
```



max = 5

```
int[] count = new int[max+1];
```

```
for (int i=0; i<n; i++) {  
    int val = arr[i];  
    count[val]++;  
}
```

count =

| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 2 | 0 | 2 | 3 | 0 | 1 |

arr[]: { ~~2~~<sup>0</sup>, ~~5~~<sup>1</sup>, ~~2~~<sup>2</sup>, ~~0~~<sup>3</sup>, ~~2~~<sup>4</sup>, ~~3~~<sup>5</sup>, ~~0~~<sup>6</sup>, ~~2~~<sup>7</sup> }

int k=0;

```
for (int i=0; i<max; i++) {  
    int c = count[i];  
    for (int j=0; j<c; j++) {  
        arr[k] = i; k++;  
    }  
}
```

i

c

0

2

1

0

2

2

3

3

4

0

5

1

→ you use Count Sort generally for char array.



## Wiggle Sort

↳ Given an unsorted array, reorder it in-place such that  $nums[0] \leq nums[1] \geq nums[2] \leq nums[3] \geq nums[4] \dots$

Ex: arr[6]: <sup>0</sup>3 <sup>1</sup>5 <sup>2</sup>2 <sup>3</sup>1 <sup>4</sup>6 <sup>5</sup>4  
↓  
{ 1 <= 6 >= 2 <= 5 >= 3 <= 4 }

## Idea 1

↳ Sort the array & then find the order

arr[6]: <sup>0</sup>3 <sup>1</sup>5 <sup>2</sup>2 <sup>3</sup>1 <sup>4</sup>6 <sup>5</sup>4  
↓  
1 2 3 4 5 6  
{ 1 3 2 5 4 6 }

T.C:  $O(N \log N)$  S.C:  $O(1)$

$nums[0] \leq nums[1] \geq nums[2] \leq nums[3] \geq nums[4] \dots$

## Idea 2

arr[9]: <sup>0</sup>8 <sup>1</sup>2 <sup>2</sup>8 <sup>3</sup>4 <sup>4</sup>10 <sup>5</sup>8 <sup>6</sup>3 <sup>7</sup>2 <sup>8</sup>15 <sup>9</sup>4  
↓  
2 8 4 10 8 3 2 15 4

$i \rightarrow \text{even} \rightarrow arr[i] \leq arr[i+1]$

$arr[i] \leq arr[i+1]$   
↳ do nothing

$arr[i] > arr[i+1]$   
↳ swap(i, i+1)

$i \rightarrow \text{odd} \rightarrow arr[i] \geq arr[i+1]$

$arr[i] \geq arr[i+1]$   
↳ do nothing

$arr[i] < arr[i+1]$   
↳ swap(i, i+1)



// Pseudo code

T.C:  $O(N)$

S.C:  $O(1)$

```
void wiggleSort (int arr[N]) {  
    for (int i = 0; i < N - 1; i++) {  
        if (i % 2 == 0) {  
            if (arr[i] > arr[i + 1]) { swap(i, i + 1); }  
        }  
        else {  
            if (arr[i] < arr[i + 1]) { swap(i, i + 1); }  
        }  
    }  
}
```



## Q) Sorting an array of objects

→ int[] arr = {10, 3, 8, 4, 7, 9}

↓ Bubble Sort  
Arrays.sort(arr);

int[] arr = {<sup>0</sup>10, <sup>1</sup>3, <sup>2</sup>8, <sup>3</sup>4, <sup>4</sup>7, <sup>5</sup>9}

3 10 10 4 10 10

{ 3, 8, 4, 7, 9, 10 }

```
public class Pair {
```

```
    int x;
```

```
    int y;
```

```
    Pair (int a, int b) {
        x = a; y = b;
    }
}
```

→ Pair[] arr = new Pair[4];

arr[0] = new Pair(10, 3);

arr[1] = new Pair(20, 6);

arr[2] = new Pair(15, 8);

arr[3] = new Pair(5, 4);

| arr | 0       | 1       | 2       | 3      |
|-----|---------|---------|---------|--------|
|     | {10, 3} | {20, 6} | {15, 8} | {5, 4} |

s.o.p (arr[2].x) → 15

s.o.p (arr[1].y) → 6

s.o.p (arr[3]) → null



|     |         |         |         |        |
|-----|---------|---------|---------|--------|
| arr | 0       | 1       | 2       | 3      |
|     | {10, 3} | {20, 6} | {15, 8} | {5, 4} |

↓

sort the array → what should be the parameter?

→ Arrays.sort(arr); → error

|     |         |         |         |        |
|-----|---------|---------|---------|--------|
| arr | 0       | 1       | 2       | 3      |
|     | {10, 3} | {20, 6} | {15, 8} | {5, 4} |

```
public class Pair implements Comparable<Pair> {
    int x;
    int y;
```

```
    Pair(int a, int b) {
        x = a; y = b;
    }
```

```
    public int compareTo(Pair o) {
        return this.y - o.y; // → inc or dec
        // → sort in inc. order on the basis of y
    }
```

```
    }

    return o.y - this.y; // → sort in dec. order on the basis of y

    return this.x - o.x; // → sort in inc. order on the basis of x
```



```
Public class Pair implements Comparable<Pair>{
```

```
    int x;
```

```
    int y;
```

```
    Pair (int a, int b){  
        x=a; y=b;  
    }
```

```
    Public int compareTo (Pair o) {  
        return this.y - o.y;  
    }
```

→ inc or dec

→ sort in inc. order on the basis of y

3

arr 0 1 2 3

|        |        |                   |                  |
|--------|--------|-------------------|------------------|
| {10,3} | {20,6} | <del>{15,8}</del> | <del>{5,4}</del> |
|        |        | {5,4}             | {15,8}           |

Arrays.sort(arr);

↳ Quick Sort



```
public static class Pair implements Comparable<Pair>{
    int x;
    int y;

    Pair(int a, int b){
        x = a;
        y = b;
    }

    public int compareTo(Pair o){
        return this.x - o.y;
    }
}

public static void main(String[] args) {
    Scanner scn = new Scanner(System.in);
    int n = scn.nextInt();
    Pair[] arr = new Pair[n];

    for(int i=0;i<n;i++){
        int a1 = scn.nextInt();
        int b1 = scn.nextInt();
        arr[i] = new Pair(a1,b1);
    }

    //System.out.println(arr[2].x); // 15
    //System.out.println(arr[3].y); // 4

    Arrays.sort(arr);

    for(int i=0;i<n;i++){
        System.out.println(arr[i].x+" "+arr[i].y);
    }
}
```



Q) Print all <sup>→ subset</sup> subsequence of given array.

Ex: arr[1] = {10}

10

—

Ex: arr[2] = {10 20}

— — { }

— 20 {20}

10 — {10}

10 20 {10 20}

Ex: arr[3] = {10 20 30}

— — — ✓ — — 30 ✓

— 20 — ✓ — 20 30 ✓

10 — — ✓ 10 — 30 ✓

10 20 — ✓ 10 20 30 ✓

→ Count of subsequence in n length array.

$$2^0 * 2^1 * 2^2 * 2^3 * 2^4 * 2^5 \approx 2^n$$

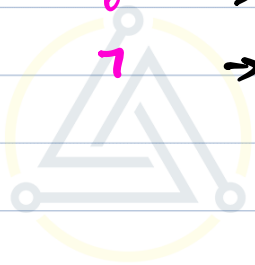


//idea

$arr[3] = \overset{0}{1} \overset{1}{10} \overset{2}{20} 30\}$

0 → reject  
1 → select

|   |   | <sup>2</sup> <sup>1</sup> <sup>0</sup> |    |    |    |     |
|---|---|----------------------------------------|----|----|----|-----|
| 0 | ~ | 0 0 0                                  |    | —  | —  | — ✓ |
| 1 | ~ | 0 0 1                                  | 10 | —  | —  | — ✓ |
| 2 | → | 0 1 0                                  | —  | 20 | —  | — ✓ |
| 3 | → | 0 1 1                                  | 10 | 20 | —  | — ✓ |
| 4 | → | 1 0 0                                  | —  | —  | 30 | — ✓ |
| 5 | → | 1 0 1                                  | 10 | —  | 30 | — ✓ |
| 6 | → | 1 1 0                                  | —  | 20 | 30 | — ✓ |
| 7 | → | 1 1 1                                  | 10 | 20 | 30 | — ✓ |



AlgoPrep



// Pseudo code

```
List < List < Integer > > Subseq (int arr[N]) {
```

```
    int totalno = (1 << N);
```

```
    List < List < Integer > > ans = new ArrayList<>();
```

```
    for (int i=0; i < totalno; i++) {
```

```
        List < Integer > temp = new ArrayList<>();
```

```
        for (int j=0; j < N; j++) {
```

```
            if (checkBit (i, j) == true) {
```

```
                temp.add(arr[j]);
```

```
            }
```

```
        }
```

```
        ans.add(temp);
```

```
    }
```

```
    return ans;
```

```
}
```

T.C:  $O(2^N \times N)$

S.C:  $O(2^N \times N)$



0 → reject  
1 → select

List < List < Integer > > Subseq (int arr[]) {

arr[] = {1, 10, 20, 30}

int totalno = (1 << n);  
List < Integer > ans = new ArrayList<>();

```
for (int i = 0; i < totalno; i++) {  
    List < Integer > temp = new ArrayList<>();  
    for (int j = 0; j < n; j++) {  
        if (checkBit (i, j) == true) {  
            temp.add (arr[j]);  
        }  
    }  
    ans.add (temp);  
}
```

return ans;

totalno = 8

|   |   |       |
|---|---|-------|
| 0 | → | 0 0 0 |
| 1 | → | 0 0 1 |
| 2 | → | 0 1 0 |
| 3 | → | 0 1 1 |
| 4 | → | 1 0 0 |
| 5 | → | 1 0 1 |
| 6 | → | 1 1 0 |
| 7 | → | 1 1 1 |

temp  
[10 20]

ans:

|   |    |    |          |
|---|----|----|----------|
| 0 | 1  | 2  | 3        |
|   | 10 | 20 | 10<br>20 |